

Skill-8

```
himasree@LAPTOP-41COT5AA:~$ cd ~/OSLab
himasree@LAPTOP-41COT5AA:~/OSLab$ nano mini_shell.c
himasree@LAPTOP-41COT5AA:~/OSLab$ gcc mini_shell.c -o mini_shell
./mini_shell
=====
          MINI OS SHELL
=====
myshell$ export NAME=Himasree
myshell$ echo $NAME
Himasree
myshell$ echo Hello $NAME
Hello Himasree
myshell$ echo ${NAME}
Himasree
myshell$ echo $UNDEFINED

myshell$ |
```

```
myshell$ pwd
/home/himasree/OSLab
myshell$ cd /tmp
myshell$ pwd
/tmp
myshell$ echo HELLO
HELLO
myshell$ export CITY=Hyderabad
myshell$ echo $CITY
Hyderabad
myshell$ unset CITY
myshell$ echo $CITY

myshell$ exit
Exiting shell...
himasree@LAPTOP-41COT5AA:~/OSLab$ |
```

```

himasree@LAPTOP-41COT5AA:~/OSLab$ cat mini_shell.c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>
#include <ctype.h>

#define MAX_INPUT 1024
#define MAX_ARGS 100

/* =====
VARIABLE EXPANSION
===== */

/* Check whether a string is a valid variable name */
int is_valid_var_char(char c) {
    return isalnum((unsigned char)c) || c == '_';
}

/* Get variable value */
const char *get_variable(const char *name) {
    const char *value = getenv(name);

    if (value == NULL)
        return "";

    return value;
}

```

```

/* Expand $VARIABLE references */
void expand_variables(char *input, char *output, size_t output_size) {
    size_t i = 0;
    size_t j = 0;

    while (input[i] != '\0' && j < output_size - 1) {

        /* Detect $ */
        if (input[i] == '$') {

            i++;

            /* Handle ${VARIABLE} */
            if (input[i] == '{') {
                i++;

                char var_name[256];
                int k = 0;

                while (input[i] != '\0' &&
                    input[i] != '}' &&
                    k < 255) {

                    var_name[k++] = input[i++];
                }

                var_name[k] = '\0';

                if (input[i] == '}')

```

```

        output[j++] = *value++;
    }
}

/* Handle $VARIABLE */
else if (isalpha((unsigned char)input[i]) ||
        input[i] == '_') {

    char var_name[256];
    int k = 0;

    while (input[i] != '\0' &&
           is_valid_var_char(input[i]) &&
           k < 255) {

        var_name[k++] = input[i++];
    }

    var_name[k] = '\0';

    const char *value = get_variable(var_name);

    while (*value != '\0' &&
           j < output_size - 1) {
        output[j++] = *value++;
    }
}

/* Just '$' */
else {

```

```

/* =====
TOKENIZATION
===== */

int tokenize(char *input, char *args[]) {
    int count = 0;

    char *token = strtok(input, " \t\n");

    while (token != NULL && count < MAX_ARGS - 1) {
        args[count++] = token;
        token = strtok(NULL, " \t\n");
    }

    args[count] = NULL;

    return count;
}

/* =====
BUILT-IN COMMANDS
===== */

```

```

/* =====
BUILT-IN COMMANDS
===== */

typedef int (*builtin_func)(char **args);

/* ----- echo ----- */
int builtin_echo(char **args) {
    for (int i = 1; args[i] != NULL; i++) {
        printf("%s", args[i]);
        if (args[i + 1] != NULL)
            printf(" ");
    }
    printf("\n");
    return 1;
}

/* ----- pwd ----- */
int builtin_pwd(char **args) {
    char cwd[1024];

```

```

    input[strcspn(input, "\n")] = '\0';

    /* Ignore empty input */
    if (strlen(input) == 0)
        continue;

    /* Variable expansion */
    expand_variables(
        input,
        expanded,
        sizeof(expanded)
    );

    /* Tokenize */
    int argc = tokenize(expanded, args);

    if (argc == 0)
        continue;

    /* Execute */
    execute_command(args);
}

return 0;
}
himasree@LAPTOP-41COT5AA:~/OSLab$ |

```